Algorithms Lab — Mid Cheat Sheet (Sorting · Graphs · Shortest Paths · MST · DP)

1. Sorting

In-place: $O(1)$ aux space. Stable: equal elements keep relative order.

Bubble Sort

Best $O(n)$, Avg/Worst $O(n^2)$, $O(1)$, stable

```
void bubbleSort(int A[], int n){
  for(int i=0;i<n-1;i++){
    bool swapped=false;
    for(int j=0;j<n-i-1;j++)
      if(A[j]>A[j+1]){
        swap(A[j],A[j+1]); swapped=true; }
    if(!swapped) break;      // already sorted
  }
}
```

Insertion Sort

Best $O(n)$, Avg/Worst $O(n^2)$, $O(1)$, stable

```
void insertionSort(int A[], int n){
  for(int i=1;i<n;i++){
    int key=A[i], j=i-1;
    while(j>=0 && A[j]>key){ A[j+1]=A[j]; j--; }
    A[j+1]=key;
  }
}
```

Selection Sort

All cases $O(n^2)$, $O(1)$, not stable

```
void selectionSort(int A[], int n){
  for(int i=0;i<n-1;i++){
    int mn=i;
    for(int j=i+1;j<n;j++) if(A[j]<A[mn]) mn=j;
    if(mn!=i) swap(A[i],A[mn]);
  }
}
```

Merge Sort

$O(n \log n)$ all cases, $O(n)$ space, stable

```
void merge(int A[],int l,int m,int r){
  vector<int> L(A+l,A+m+1), R(A+m+1,A+r+1);
  int i=0,j=0,k=l;
  while(i<L.size() && j<R.size())
    A[k++] = (L[i]<=R[j]) ? L[i++] : R[j++];
  while(i<L.size()) A[k++]=L[i++];
  while(j<R.size()) A[k++]=R[j++];
}
void mergeSort(int A[],int l,int r){
  if(l<r){
    int m=l+(r-l)/2;
    mergeSort(A,l,m); mergeSort(A,m+1,r);
    merge(A,l,m,r);
  }
}
```

Counting Sort

$O(n+m)$ time & space, stable (m=maxVal)

```
vector<int> countingSort(vector<int>& A){
  int n=A.size(), mx=*max_element(A.begin(),A.end());
  vector<int> cnt(mx+1,0), res(n);
  for(int x:A) cnt[x]++;              // frequency
  for(int i=1;i<=mx;i++) cnt[i]+=cnt[i-1]; // prefix
  for(int i=n-1;i>=0;i--)             // stable build
    res[--cnt[A[i]]]=A[i];
  return res;
}
```

Radix Sort

$O(d(n+b))$, $O(n+b)$ space (b=base,d=digits)

```
void countByDigit(vector<int>& A,int exp){
  int n=A.size();
  vector<int> cnt(10,0), res(n);
  for(int x:A) cnt[(x/exp)%10]++;
  for(int i=1;i<10;i++) cnt[i]+=cnt[i-1];
  for(int i=n-1;i>=0;i--){
    int d=(A[i]/exp)%10; res[--cnt[d]]=A[i]; }
  A=res;
}
void radixSort(vector<int>& A){
  int mx=*max_element(A.begin(),A.end());
  for(int exp=1; mx/exp>0; exp*=10)
    countByDigit(A,exp);
}
```

2. Graph Traversal

AdjMatrix: space $O(V^2)$, edge-check $O(1)$. AdjList: space $O(V+E)$.

BFS — Adjacency List

$O(V+E)$ time, $O(V)$ space

```
void bfsList(vector<int> adj[], int V, int src){
  vector<bool> vis(V,false);
  queue<int> q; vis[src]=true; q.push(src);
  while(!q.empty()){
    int u=q.front(); q.pop(); cout<<u<<" ";
    for(int v:adj[u])
      if(!vis[v]){ vis[v]=true; q.push(v); }
  }
}
```

BFS — Adjacency Matrix

$O(V^2)$ time, $O(V)$ space

```
void bfsMatrix(vector<vector<int>>& g, int src){
  int V=g.size();
  vector<bool> vis(V,false);
  queue<int> q; vis[src]=true; q.push(src);
  while(!q.empty()){
    int u=q.front(); q.pop(); cout<<u<<" ";
    for(int i=0;i<V;i++)          // scan whole row
      if(g[u][i] && !vis[i]){ vis[i]=true; q.push(i); }
  }
}
```

DFS (Recursive)

$O(V+E)$ time, $O(V)$ space

```
void dfs(vector<int> adj[], vector<bool>& vis, int u){
  vis[u]=true; cout<<u<<" ";
  for(int v:adj[u]) if(!vis[v]) dfs(adj,vis,v);
}
```

Topological Sort (DFS)

$O(V+E)$, DAG only

```
void topoDFS(vector<int> adj[], vector<bool>& vis,
             stack<int>& st, int u){
  vis[u]=true;
  for(int v:adj[u]) if(!vis[v]) topoDFS(adj,vis,st,v);
  st.push(u);              // push AFTER all neighbors
}
void topoSort(vector<int> adj[], int V){
  vector<bool> vis(V,false); stack<int> st;
  for(int i=0;i<V;i++) if(!vis[i]) topoDFS(adj,vis,st,i);
  while(!st.empty()){ cout<<st.top()<<" "; st.pop(); }
}
```

3. Shortest Paths

Dijkstra (SSSP)

$O((V+E)\log V)$; non-negative weights

```
// adj[u] = list of {v, weight}
vector<int> dijkstra(
    vector<vector<pair<int,int>>>& adj, int V, int src){
  vector<int> dist(V,INT_MAX);
  vector<bool> vis(V,false);
  priority_queue<pair<int,int>,
    vector<pair<int,int>>,greater<>> pq;
  dist[src]=0; pq.push({0,src});
  while(!pq.empty()){
    auto [d,u]=pq.top(); pq.pop();
    if(vis[u]) continue;      // lazy deletion
    vis[u]=true;
    for(auto [v,w]:adj[u])
      if(!vis[v] && d+w<dist[v]){
        dist[v]=d+w; pq.push({dist[v],v}); }
  }
  return dist;
}
```

Bellman-Ford (SSSP)

$O(V \cdot E)$; handles negative edges

```
// edges: list of {u, v, weight}
vector<int> bellmanFord(int V,
    vector<array<int,3>>& edges, int src){
  vector<int> dist(V,1e9); dist[src]=0;
  for(int i=0;i<V;i++)          // V-1 relax + 1 check
    for(auto& e:edges){
      int u=e[0], v=e[1], w=e[2];
      if(dist[u]!=1e9 && dist[u]+w<dist[v]){
        if(i==V-1) return {-1}; // negative cycle
        dist[v]=dist[u]+w;
      }
    }
  return dist;
}
```

Floyd-Warshall (APSP)

$O(V^3)$ time, $O(V^2)$ space

```
// dist init: 0 on diagonal, w for edges, INF else
void floydWarshall(vector<vector<int>>& dist, int V){
  for(int k=0;k<V;k++)          // intermediate vertex
    for(int i=0;i<V;i++)        // source
      for(int j=0;j<V;j++)      // destination
        if(dist[i][k]!=INF && dist[k][j]!=INF)
          dist[i][j]=min(dist[i][j],
                         dist[i][k]+dist[k][j]);
}
// dist[i][i] < 0 after run  =>  negative cycle
```

4. Minimum Spanning Tree

MST edge choice: $w(u,v) < cost[v]$ (vs Dijkstra $cost[u]+w < cost[v]$).

Prim's Algorithm

$O((V+E)\log V)$, $O(V+E)$ space

```
// adj[u] = list of {v, weight}
int primMST(vector<vector<pair<int,int>>>& adj,
            int V, int src){
  vector<int> cost(V,INT_MAX);
  vector<bool> vis(V,false);
  priority_queue<pair<int,int>,
    vector<pair<int,int>>,greater<>> pq;
  cost[src]=0; pq.push({0,src}); int res=0;
  while(!pq.empty()){
    auto [wt,u]=pq.top(); pq.pop();
    if(vis[u]) continue;
    vis[u]=true; res+=wt;       // add edge to MST
    for(auto [v,w]:adj[u])
      if(!vis[v] && w<cost[v]){
        cost[v]=w; pq.push({w,v}); }
  }
  return res;                   // total MST weight
}
```

DSU (Union by Rank + Path Compression)

$\approx O(\alpha)$/op

```
class DSU {
  vector<int> parent, rnk;
public:
  DSU(int n){
    parent.resize(n); rnk.resize(n,1);
    for(int i=0;i<n;i++) parent[i]=i;
  }
  int find(int i){              // path compression
    return parent[i]==i ? i
                        : parent[i]=find(parent[i]);
  }
  void unite(int x,int y){      // union by rank
    int a=find(x), b=find(y);
    if(a==b) return;
    if(rnk[a]<rnk[b]) parent[a]=b;
    else if(rnk[a]>rnk[b]) parent[b]=a;
    else { parent[b]=a; rnk[a]++; }
  }
};
```

Kruskal's Algorithm

$O(E \log V)$, $O(V+E)$ space

```
// edges[i] = {u, v, weight}
int kruskalMST(int V, vector<vector<int>>& edges){
  sort(edges.begin(), edges.end(),
    [](auto& a, auto& b){ return a[2]<b[2]; });
  DSU dsu(V); int cost=0, count=0;
  for(auto& e:edges){
    int x=e[0], y=e[1], w=e[2];
    if(dsu.find(x)!=dsu.find(y)){  // no cycle
      dsu.unite(x,y); cost+=w;
      if(++count==V-1) break;
    }
  }
  return cost;
}
```

5. Dynamic Programming

Needs: overlapping subproblems + optimal substructure.

Fibonacci — Memoization

$O(n)$ time, $O(n)$ space

```
// vector<int> dp(n+1,-1);
int fibMemo(int n, vector<int>& dp){
  if(n<=1) return n;
  if(dp[n]!=-1) return dp[n];
  return dp[n]=fibMemo(n-1,dp)+fibMemo(n-2,dp);
}
```

Fibonacci — Tabulation

$O(n)$ time, $O(n)$ space

```
int fibTab(int n){
  if(n<=1) return n;
  vector<int> dp(n+1); dp[0]=0; dp[1]=1;
  for(int i=2;i<=n;i++) dp[i]=dp[i-1]+dp[i-2];
  return dp[n];
}
```

Fibonacci — Space Optimized

$O(n)$ time, $O(1)$ space

```
int fibOpt(int n){
  if(n<=1) return n;
  int a=0, b=1;
  for(int i=2;i<=n;i++){ int c=a+b; a=b; b=c; }
  return b;
}
```

Coin Change 1 — Min Coins

$O(N \cdot C)$ time, $O(N)$ space

$dp[x] = \min_c(dp[x-c]) + 1$

```
// Memoization  (INF = 1e9)
int coinMinMemo(int x, vector<int>& coins,
                vector<int>& dp){
  if(x==0) return 0;
  if(x<0)  return INF;
  if(dp[x]!=-1) return dp[x];
  int best=INF;
  for(int c:coins){
    int s=coinMinMemo(x-c,coins,dp);
    if(s!=INF) best=min(best,s+1);
  }
  return dp[x]=best;
}
// Tabulation
int coinMinTab(vector<int>& coins, int target){
  vector<int> dp(target+1,1e9); dp[0]=0;
  for(int i=1;i<=target;i++)
    for(int c:coins) if(i>=c)
      dp[i]=min(dp[i],dp[i-c]+1);
  return dp[target];
}
```

Coin Change 2 — Ways (Permutations)

$O(N \cdot C)$, order matters

$dp[x] = \sum_c dp[x-c]$; *target-loop OUTER, coin-loop INNER*

```
// Memoization
long long permMemo(int x, vector<int>& coins,
                   vector<long long>& dp){
  if(x==0) return 1;
  if(x<0)  return 0;
  if(dp[x]!=-1) return dp[x];
  long long ways=0;
  for(int c:coins) ways+=permMemo(x-c,coins,dp);
  return dp[x]=ways;
}
// Tabulation
long long permTab(vector<int>& coins, int target){
  vector<long long> dp(target+1,0); dp[0]=1;
```

```
  for(int i=1;i<=target;i++)
    for(int c:coins) if(i>=c) dp[i]+=dp[i-c];
  return dp[target];
}
```

Coin Change 3 — Ways (Combinations)

$O(N \cdot C)$, order ignored

coin-loop OUTER, target-loop INNER (forces order $\Rightarrow$ unique sets)

```
// Memoization (2D: needs coin index)
long long combMemo(int idx, int x, vector<int>& coins,
                   vector<vector<long long>>& dp){
  if(x==0) return 1;
  if(idx>=(int)coins.size() || x<0) return 0;
  if(dp[idx][x]!=-1) return dp[idx][x];
  long long take=combMemo(idx,x-coins[idx],coins,dp);
  long long skip=combMemo(idx+1,x,coins,dp);
  return dp[idx][x]=take+skip;
}
// Tabulation
long long combTab(vector<int>& coins, int target){
  vector<long long> dp(target+1,0); dp[0]=1;
  for(int c:coins)
    for(int s=c;s<=target;s++) dp[s]+=dp[s-c];
  return dp[target];
}
```

Friends Pairing

$O(n)$ time; $dp[i] = dp[i-1] + (i-1)dp[i-2]$

```
// Memoization
int friendsMemo(int n, vector<int>& dp){
  if(n<=2) return n;
  if(dp[n]!=-1) return dp[n];
  return dp[n]=friendsMemo(n-1,dp)
              +(n-1)*friendsMemo(n-2,dp);
}
// Tabulation
int friendsTab(int n){
  vector<int> dp(n+1);
  for(int i=0;i<=n;i++)
    dp[i] = (i<=2) ? i
                   : dp[i-1]+(i-1)*dp[i-2];
  return dp[n];
}
```

Climbing Stairs (1/2/3 steps)

$O(n)$; $dp[i] = dp[i-1]+dp[i-2]+dp[i-3]$

```
// Memoization
int climbMemo(int n, vector<int>& dp){
  if(n<=1) return 1;
  if(n==2) return 2;
  if(dp[n]!=-1) return dp[n];
  return dp[n]=climbMemo(n-1,dp)
              +climbMemo(n-2,dp)+climbMemo(n-3,dp);
}
// Tabulation
int climbTab(int n){
  if(n<=1) return 1;
  if(n==2) return 2;
  vector<int> dp(n+1); dp[0]=1; dp[1]=1; dp[2]=2;
  for(int i=3;i<=n;i++)
    dp[i]=dp[i-1]+dp[i-2]+dp[i-3];
  return dp[n];
}
```